C:\FFGPT · PROYECTO EDUCATIVO

Entendiendo mini-gpt

Guía paso a paso del código: qué hace cada módulo, por qué existe, qué forma tienen los tensores, y cómo recorrerlo vos mismo.

Un GPT decoder-only construido 100% desde cero en PyTorch
30 módulos · 235 tests · sin Hugging Face transformers, sin nanoGPT copiado

Tabla de contenido

0. Cómo usar esta guía (el ritual para leer el código)

1. Cimientos: tokenizers y dataset

- 1.1 Interfaz común (`Tokenizer`)
- 1.2 CharTokenizer
- 1.3 BPETokenizer
- 1.4 TokenDataset y DataLoader

2. Representación: embeddings y posición

- 2.1 TokenEmbedding
- 2.2 Positional Encoding (senoidal y aprendida)

3. El corazón: Self-Attention

- 3.1 SelfAttention (una sola cabeza)
- 3.2 MultiHeadAttention

4. El bloque Transformer

- 4.1 LayerNorm
- 4.2 FeedForward y GELU
- 4.3 ResidualConnection (Pre-LN)
- 4.4 DecoderBlock
- 4.5 TransformerStack
- 4.6 OutputHead
- 4.7 GPTModel — todo ensamblado

5. Entrenamiento

- 5.1 CrossEntropyLoss
- 5.2 Trainer (el loop)
- 5.3 AdamW desde cero
- 5.4 WarmupCosineScheduler
- 5.5 Checkpoints (guardar / cargar)
- 5.6 Fine-tuning

6. Inferencia y generación

- 6.1 Generator (el loop autoregresivo)
- 6.2 Sampling: temperature, top-k, top-p
- 6.3 Visualización de attention
- 6.4 ChatSession

7. Mapa completo de los 30 módulos

8. Decisiones de diseño que vale la pena notar

9. Cómo seguir explorando por tu cuenta

0. Cómo usar esta guía

Este proyecto no es un solo notebook: son **30 módulos** chiquitos, cada uno con su código en `app/`, su test en `tests/`, y un script demo corriendo en `scripts/`. Esa estructura ya te da el método de estudio.

Ritual sugerido, por módulo:

1. **Leé el docstring del archivo fuente primero.** Cada archivo en `app/` explica la teoría y la fórmula matemática arriba de la clase, antes del código — no hace falta buscar la teoría en otro lado.
2. **Corré el demo:** `uv run python scripts/<modulo>_demo.py`. Vas a ver tensores reales con shapes reales, no solo teoría abstracta.
3. **Mirá el test** en `tests/test_<modulo>.py` si algo no cerró — los tests son casos de uso concretos y muchos comparan contra la implementación oficial de PyTorch (LayerNorm, GELU, AdamW, cross-entropy), así que también te confirman que "lo hecho a mano" da lo mismo que "lo hecho por la librería".

Esta guía sigue el mismo orden en que se construyó el proyecto (ver la tabla completa en la sección 7), pero agrupado en **6 grandes etapas** — el mismo recorrido conceptual que vas a reconocer de cualquier explicación de Transformers: texto → representación → attention → bloque completo → entrenamiento → inferencia.

💡 Tip: `grep -n "Module" app/**/*.py` te muestra las referencias cruzadas entre módulos que quedaron documentadas en los docstrings (ej. "Module 19" mencionado desde `self_attention.py`) — es un mapa de dependencias gratis.

1. Cimientos: tokenizers y dataset

Antes de que exista cualquier red neuronal, el texto tiene que volverse números. Esta etapa cubre esa conversión y cómo se arman los batches de entrenamiento.

1.1 · Interfaz común

`app/tokenizer/base.py`

Un `Protocol` de Python (no una clase base con herencia) que define el contrato que cualquier tokenizer debe cumplir: `vocab_size`, `encode(text) -> list[int]`, `decode(ids) -> str`.

> **Por qué existe:** el resto del proyecto (dataset, generator) depende de esta interfaz, no de si el tokenizer es por carácter o por BPE. Podés cambiar de tokenizer sin tocar nada más.

1.2 · CharTokenizer

`app/tokenizer/char_tokenizer.py · scripts/tokenizer_demo.py`

El más simple posible: agarra un corpus, saca el conjunto de caracteres distintos (`sorted(set(text))`) y le asigna un id a cada uno. `encode` / `decode` son lookups directos en un diccionario (`stoi` / `itos`).

> **Por qué existe:** es el tokenizer que usa `scripts/train_tiny_shakespeare.py`. Vocabulario chico (~65 caracteres), fácil de razonar, ideal para ver el pipeline completo funcionando rápido. Limitación real: si aparece un carácter que no vio al entrenar, `encode` tira `ValueError` — no tiene fallback.

1.3 · BPETokenizer

`app/tokenizer/bpe.py · scripts/bpe_demo.py`

Byte Pair Encoding, el mismo algoritmo detrás de GPT-2/3. Vocabulario base = 256 bytes posibles, así que **nunca** falla por carácter desconocido (a diferencia del CharTokenizer). `train()` busca repetidamente el par de tokens adyacentes más frecuente y lo fusiona en uno nuevo, hasta llegar a `vocab_size`.

> **Por qué existe:** es el tokenizer que usan los LLMs reales — le permite representar cualquier texto UTF-8 con un vocabulario mucho más chico que "una palabra = un token", sin perder la capacidad de caer a bytes crudos como último recurso.

1.4 · TokenDataset + DataLoader

`app/dataset/token_dataset.py, app/dataset/dataloader.py · scripts/dataset_demo.py`

Dado un stream plano de ids, `TokenDataset` devuelve ventanas de `block_size` tokens como input, y esa misma ventana corrida un token a la derecha como target — el clásico setup de *next-token prediction* con *teacher forcing*. `create_dataloader` solo envuelve eso en un `torch.utils.data.DataLoader` para batchear.

> **Shapes:** cada item es `(block_size,)`; cada batch, `(batch_size, block_size)`. Este es el mismo formato de `X` / `Y` que viste armado a mano en el notebook de Colab — acá se generaliza a cualquier corpus.

2. Representación: embeddings y posición

Los ids de token son solo etiquetas arbitrarias sin significado. Esta etapa los convierte en vectores que sí puede aprender la red, y les agrega noción de orden.

2.1 · TokenEmbedding

`app/embeddings/token_embedding.py · scripts/embedding_demo.py`

Una tabla de lookup entrenable: `nn.Embedding(vocab_size, d_model)`, inicializada con `std=0.02` (la convención de GPT-2, para arrancar con magnitudes chicas y estables).

> **Importante:** el mismo id siempre mapea al mismo vector, sin importar en qué posición de la secuencia aparece — todavía no sabe nada de orden. Eso es trabajo del siguiente módulo.

2.2 · Positional Encoding

`app/embeddings/positional_encoding.py · scripts/positional_encoding_demo.py`

Hay **dos** implementaciones:

- `SinusoidalPositionalEncoding`: fórmula fija con senos/cosenos de distinta frecuencia por dimensión. No tiene parámetros entrenables, así que generaliza a secuencias más largas que las vistas en entrenamiento.
- `LearnedPositionalEncoding`: una tabla entrenable igual que `TokenEmbedding`, pero indexada por posición en vez de por token. **Esta es la que usa `GPTModel`** (la misma convención de GPT-2) — más flexible, pero no generaliza más allá de `max_seq_len`.

> **Cómo se combinan:** `x = TokenEmbedding(ids) + PositionalEncoding(seq_len)` — una suma elemento a elemento, exactamente igual que en el notebook de Colab.

3. El corazón: Self-Attention

El mecanismo que le permite a cada token "mirar" a los demás tokens de la secuencia (los anteriores, nunca los futuros) para construir su representación.

3.1 · SelfAttention (una cabeza)

`app/attention/self_attention.py` · `scripts/self_attention_demo.py`

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d_head) + M) V
```

Proyecta `x` a Q, K y V con tres `nn.Linear` independientes. Calcula `scores = Q @ K.T / sqrt(d_head)`, aplica una **máscara causal** triangular inferior (`torch.tril`) que fuerza `-inf` en las posiciones futuras antes del softmax, y multiplica los pesos resultantes por `V`.

> **La máscara causal es lo que hace posible GPT:** en inferencia, los tokens futuros literalmente no existen todavía — la máscara solo hace que el entrenamiento sea consistente con eso.

3.2 · MultiHeadAttention

`app/attention/multi_head_attention.py` · `scripts/multi_head_attention_demo.py`

En vez de una sola atención sobre todo `d_model`, divide en `n_heads` subespacios de `d_head = d_model / n_heads`, corre attention causal en cada uno en paralelo, y concatena + proyecta el resultado con `output_proj`.

> **Por qué:** cada cabeza puede especializarse en un patrón distinto (una en "el token anterior", otra en dependencias de largo alcance) en vez de promediar todo en un solo patrón. Guarda además `last_attention_weights` como atributo — un "gancho" que después usa el módulo de visualización (sección 6.3) sin tener que tocar esta clase.

4. El bloque Transformer

Attention por sí sola no alcanza. Esta etapa arma el bloque completo (attention + procesamiento por posición + normalización + conexiones residuales) y lo apila para construir el modelo entero.

4.1 · LayerNorm

`app/transformer/layernorm.py · scripts/layernorm_demo.py`

```
y = gamma * (x - mean) / sqrt(var + eps) + beta
```

Implementado a mano (no `torch.nn.LayerNorm`), normaliza cada token de forma independiente sobre sus `d_model` features — sin depender del batch ni de otras posiciones, por eso se comporta igual en training y en inferencia.

4.2 · FeedForward + GELU

`app/transformer/feedforward.py · scripts/feedforward_demo.py`

```
FFN(x) = Linear2(GELU(Linear1(x)))   —   d_model → 4·d_model → d_model
```

`GELU` también está implementada desde cero (con la forma exacta vía `torch.erf`, y una variante aproximada con `tanh` igual a la de GPT-2 original). A diferencia de attention, esta capa procesa cada posición de forma **independiente** — nunca mezcla información entre tokens.

4.3 · ResidualConnection (Pre-LN)

`app/transformer/residual.py · scripts/residual_connection_demo.py`

```
output = x + dropout(sublayer(LayerNorm(x)))
```

> **Diferencia clave con el notebook de Colab:** ahí el LayerNorm se aplicaba *después* de sumar el residual (Post-LN, el paper original de 2017). Acá se aplica *antes* de entrar a la sublayer (Pre-LN, la convención de GPT-2) — entrena más estable en redes profundas porque el "camino residual" nunca se normaliza directamente, y el gradiente tiene un camino directo (`d(output)/dx = I + d(sublayer)/dx`) que ninguna sublayer puede bloquear del todo.

4.4 · DecoderBlock

`app/transformer/decoder_block.py · scripts/decoder_block_demo.py`

```
x = x + MultiHeadAttention(LayerNorm(x))
x = x + FeedForward(LayerNorm(x))
```

Un bloque = dos `ResidualConnection` independientes, una envolviendo attention y otra envolviendo el feed-forward. Esta es la unidad que GPT apila muchas veces.

4.5 · TransformerStack

`app/transformer/transformer_stack.py · scripts/transformer_stack_demo.py`

`n_layers` bloques `DecoderBlock` en secuencia (`nn.ModuleList`), seguidos de **un LayerNorm final**.

> **Por qué el LayerNorm final:** como todos los bloques son Pre-LN, la salida del último bloque es una suma acumulada sin normalizar de N bloques. Este LayerNorm la vuelve a poner en una escala consistente justo antes de la proyección de salida.

4.6 · OutputHead

`app/transformer/output_head.py · scripts/output_head_demo.py`

```
logits = h @ W_out^T   —   (batch, seq_len, d_model) → (batch, seq_len, vocab_size)
```

> **Weight tying:** opcionalmente, `W_out` es literalmente el mismo tensor de parámetros que `TokenEmbedding` (no una copia). Ahorra `vocab_size × d_model` parámetros — la matriz más grande de un GPT chico — y actúa como regularizador leve. Es la convención de GPT-2 y el default de este proyecto.

4.7 · GPTModel — todo ensamblado

`app/transformer/gpt_model.py`

```
logits = OutputHead( TransformerStack( TokenEmbedding(ids) + PositionalEncoding(ids) ) )
```

Acá se conectan, en un solo `nn.Module`, todos los módulos de las etapas 2 a 4. Es el objeto que `Trainer` (etapa 5) entrena y que `Generator` (etapa 6) usa para generar texto.

> **Comparación directa con el notebook de Colab:** es exactamente el mismo `TinyTransformer` que viste ahí, pero con Pre-LN en vez de Post-LN, GELU en vez de ninguna elección explícita, weight tying, y separado en módulos reales en vez de todo en una celda.

5. Entrenamiento

Con el modelo armado, esta etapa cubre cómo se mide el error, cómo se actualizan los parámetros, y cómo se persiste el progreso.

5.1 · CrossEntropyLoss

`app/training/loss.py · scripts/loss_demo.py`

```
loss = -log( softmax(logits)[target] )
```

Implementada a mano con el truco log-sum-exp (restar el máximo antes de exponenciar) para estabilidad numérica — matemáticamente idéntica a `nn.CrossEntropyLoss`, pero visible paso a paso. Aplana `(batch, seq_len, vocab_size)` a `(batch·seq_len, vocab_size)` y promedia sobre todas las posiciones.

5.2 · Trainer

`app/training/trainer.py · scripts/training_loop_demo.py`

`train_step` hace exactamente el ciclo: `forward → loss → zero_grad → backward → optimizer.step()`. `fit` lo repite hasta `max_steps`, re-iterando el `DataLoader` si se agota antes, y actualizando el learning rate en cada paso si le pasás un `scheduler`.

5.3 · AdamW desde cero

`app/training/optimizer.py · scripts/optimizer_demo.py`

```
m_t = β1·m_{t-1} + (1-β1)·g_t          (promedio móvil del gradiente)
v_t = β2·v_{t-1} + (1-β2)·g_t²         (promedio móvil del gradiente²)
θ_t = θ_{t-1}·(1 - lr·wd) - lr·m̂_t/(√v̂_t + ε)
```

> **La "W" de decoupled weight decay:** a diferencia de Adam + L2 clásico (que mete el weight decay dentro del gradiente, donde termina distorsionado por el escalado adaptativo $1/\sqrt{\hat{v}}$), acá el weight decay es un paso *separado* aplicado directo sobre los parámetros. `build_param_groups` además excluye de decay a los parámetros 1D (biases, gamma/beta de LayerNorm) — decaer el gamma de LayerNorm (que arranca en 1) iría en contra de su propósito.

5.4 · WarmupCosineScheduler

`app/training/scheduler.py · scripts/scheduler_demo.py`

Sube el learning rate linealmente desde 0 durante `warmup_steps` (evita pasos grandes y desestabilizantes mientras los momentos de AdamW todavía no son confiables), y después decae con una curva de coseno hasta `min_lr`. Es el schedule que usan GPT-2/3 para pre-entrenar.

5.5 · Checkpoints

`app/training/checkpoint.py · scripts/checkpoint_demo.py, checkpoint_resume_demo.py`

`save_checkpoint` guarda el `state_dict` del modelo, el del optimizador (los buffers m/v de AdamW — sin esto, resumir reiniciaría esas estadísticas de golpe), el `step` actual, y un `ModelConfig` con los hiperparámetros necesarios para reconstruir el modelo. `load_checkpoint` lo reconstruye todo.

> **Detalle no obvio:** si le pasás un `optimizer` para resumir entrenamiento, **tenés** que pasarle también el `model` exacto sobre el que ese optimizer fue construido — sus grupos de parámetros referencian tensores específicos por identidad, no por nombre.

5.6 · Fine-tuning

`app/training/finetune.py · scripts/finetune_demo.py`

`freeze_all_parameters` pone `requires_grad=False` en todo. `unfreeze_last_n_blocks` reactiva solo los últimos N bloques decoder + el output head — la idea de que las capas tempranas capturan patrones generales reutilizables, y solo las últimas necesitan adaptarse a una tarea nueva.

> **Trampa si usás weight tying:** si el modelo tiene `tie_weights=True`, el output head y el token embedding son *el mismo* parámetro — descongelar el output head descongela también el embedding, aunque pongas `n_trainable_blocks=0`.

6. Inferencia y generación

Con un modelo entrenado, esta etapa cubre cómo generar texto token por token, cómo controlar la aleatoriedad de esa generación, y cómo mirar "adentro" del modelo mientras genera.

6.1 · Generator

`app/inference/generator.py · scripts/generation_demo.py`

El loop autoregresivo: en cada paso corre el modelo completo sobre la secuencia actual (truncada a `max_seq_len`), toma los logits de la **última** posición, elige un token vía `sample_fn` (por default, greedy/argmax), lo agrega, y repite.

> ⚠️ Nota de rendimiento documentada en el propio código: recalcula el forward pass completo en cada paso — `O(nuevos_tokens · seq_len²)` — porque no implementa **KV cache** (cachear las proyecciones K/V ya calculadas). Es la optimización natural que le falta, dejada afuera a propósito para mantener el loop legible.

6.2 · Sampling: temperature, top-k, top-p

`app/inference/sampling.py, temperature.py, top_k.py, top_p.py`

Se combinan en este orden antes de elegir un token: **temperature** (`logits / T`: T<1 afila la distribución, T>1 la achata, sin cambiar el orden) → **top-k** (deja vivos solo los K logits más altos, el resto a `-inf`) → **top-p / nucleus** (deja vivo el conjunto más chico de tokens cuya probabilidad acumulada llega a `p` — a diferencia de top-k, el tamaño de ese conjunto se adapta solo según qué tan segura está la distribución) → `sample_from_logits` (softmax + `torch.multinomial`, en vez de argmax).

6.3 · Visualización de attention

`app/inference/attention_viz.py · scripts/attention_viz_demo.py`

`extract_attention_weights` corre un forward pass y recolecta `last_attention_weights` de cada `DecoderBlock` (el "gancho" que dejó `MultiHeadAttention`, sección 3.2) sin tener que modificar el modelo. `plot_attention_heatmap` guarda un PNG — la misma matriz Query × Key que viste como `imshow` en el notebook de Colab, pero para un modelo real entrenado.

6.4 · ChatSession

`app/inference/chat.py · scripts/chat.py, web_chat.py`

Mantiene un `transcript` de texto acumulado y genera continuaciones sobre él, aplicando la config de sampling (`SamplingConfig` : temperature=0.8, top_p=0.9 por default) en cada paso.

> **Importante — no es ChatGPT:** el modelo detrás (entrenado sobre Tiny Shakespeare) nunca vio datos de pregunta/respuesta ni pasó por RLHF. `ChatSession` solo hace lo mismo que `Generator` : completa texto. Escribirle algo no te da una "respuesta", te da una continuación estilo Shakespeare de lo que escribiste.

7. Mapa completo de los 30 módulos

El orden real de construcción (de `README.md`), por si querés ir módulo por módulo en vez de por etapas.

#	Módulo	Código	Demo
1	Setup del proyecto	—	check_environment.py
2	Tokenizer por carácter	tokenizer/char_tokenizer.py	tokenizer_demo.py
3	Tokenizer BPE	tokenizer/bpe.py	bpe_demo.py
4	Dataset loader	dataset/	dataset_demo.py
5	Embedding layer	embeddings/token_embedding.py	embedding_demo.py
6	Positional encoding	embeddings/positional_encoding.py	positional_encoding_demo.py
7	LayerNorm	transformer/layernorm.py	layernorm_demo.py
8	Feed forward	transformer/feedforward.py	feedforward_demo.py
9	Self attention	attention/self_attention.py	self_attention_demo.py
10	Multi-head attention	attention/multi_head_attention.py	multi_head_attention_demo.py
11	Residual connections	transformer/residual.py	residual_connection_demo.py
12	Decoder block	transformer/decoder_block.py	decoder_block_demo.py
13	Transformer stack	transformer/transformer_stack.py	transformer_stack_demo.py
14	Output projection	transformer/output_head.py, gpt_model.py	output_head_demo.py
15	Loss function	training/loss.py	loss_demo.py
16	Training loop	training/trainer.py	training_loop_demo.py
17	Optimizer	training/optimizer.py	optimizer_demo.py
18	LR scheduler	training/scheduler.py	scheduler_demo.py
19	Generación de texto	inference/generator.py	generation_demo.py
20	Sampling	inference/sampling.py	sampling_demo.py
21	Temperature	inference/temperature.py	temperature_demo.py
22	Top-K sampling	inference/top_k.py	top_k_demo.py
23	Top-P sampling	inference/top_p.py	top_p_demo.py
24	Guardar checkpoints	training/checkpoint.py	checkpoint_demo.py
25	Cargar checkpoints	training/checkpoint.py	checkpoint_resume_demo.py
26	Fine-tuning	training/finetune.py	finetune_demo.py
27	Visualizar attention	inference/attention_viz.py	attention_viz_demo.py

28	Entrenar en Tiny Shakespeare	—	train_tiny_shakespeare.py
29	Entrenar en WikiText	—	train_wikitext.py
30	Interfaz de chat	inference/chat.py	chat.py

8. Decisiones de diseño que vale la pena notar

- **Pre-LN, no Post-LN** — LayerNorm antes de cada sublayer, no después de sumar el residual. Más estable en modelos profundos (sección 4.3).
- **Weight tying activado por default** — el embedding y la proyección de salida comparten parámetros (sección 4.6).
- **AdamW y el scheduler coseno están hechos a mano**, no son `torch.optim.AdamW` — y están cruzados contra la implementación oficial de PyTorch en los tests.
- **Sin KV cache** — decisión consciente de simplicidad sobre performance en `Generator` (sección 6.1).
- **CharTokenizer vs. BPETokenizer** — el proyecto tiene los dos; Tiny Shakespeare usa char-level (vocabulario chico, loop rápido de ver resultados), pero BPE es el camino a un tokenizer que nunca falla por carácter desconocido.

9. Cómo seguir explorando por tu cuenta

1. Corré `uv run pytest -v` y mirá los nombres de los 235 tests — son básicamente una lista de "cosas que este código garantiza que son ciertas".
2. Entrená el modelo chico: `uv run python scripts/train_tiny_shakespeare.py` (~9 min en CPU), y después `uv run python scripts/chat.py` para hablar con él.
3. Corré `scripts/attention_viz_demo.py` y mirá el PNG resultante — es la forma más directa de *ver* qué está mirando cada cabeza de attention, en vez de solo leerlo.
4. Ejercicio del notebook de Colab que sigue aplicando acá: sacá temporalmente la máscara causal de `SelfAttention` y pensá por qué eso rompería el entrenamiento para next-token prediction.
5. Cuando quieras seguir construyendo en vez de solo leer, decime y retomamos el ritual módulo por módulo (teoría → shapes → implementación → tests → demo) para lo que venga después de los 30 módulos actuales.

Generado a partir del código fuente de C:\ffgpt el 2026-08-20. Los docstrings citados vienen directo de app/**/*.py y README.md — si el código cambia, esta guía puede quedar desactualizada; regenerarla es la forma más confiable de mantenerla al día.